

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

โปรเจกต์ Game Catalog นี้ถูกออกแบบตามแนวคิด Layered Architecture โดยแบ่งความรับผิดชอบออกเป็น 4 ชั้นหลัก ได้แก่ Entity (Model), Repository, Service และ Controller ซึ่งการแบ่งชั้นนี้สอดคล้องกับหลักการ GRASP (General Responsibility Assignment Software Patterns) ดังนี้

1.Controller Pattern

คลาส GameController ทำหน้าที่เป็นตัวกลางรับ HTTP Request จากผู้ใช้ (Browser/Form) แล้วส่งต่อให้ GameService จัดการ โดย Controller เองไม่มี business logic ใด ๆ ปะปนอยู่ มีหน้าที่เพียงรับข้อมูล เรียกใช้ Service Layer และเลือก View (Thymeleaf template) หรือ redirect กลับไปยังหน้าที่เหมาะสม ทำให้ Controller มีความรับผิดชอบที่ชัดเจนตามหลัก Controller Pattern

2.High Cohesion

แต่ละคลาสในระบบมีความสอดคล้องภายในสูงคือทุก method ภายในคลาสเดียวกันล้วนเกี่ยวข้องกับหน้าที่เดียวกัน เช่น GameService มีเฉพาะ method ที่เกี่ยวกับการจัดการข้อมูลเกม ไม่มี logic ของการ render หน้าเว็บหรือการเข้าถึงฐานข้อมูลโดยตรงปะปนอยู่

3.Low Coupling

การเชื่อมโยงระหว่างคลาสถูกออกแบบให้หลวมโดยใช้ Dependency Injection ผ่าน Constructor เช่น GameController รู้จักเพียง GameService เท่านั้น ไม่รู้จัก GameRepository หรือ DiscountContext โดยตรง การเปลี่ยนแปลงภายใน Repository หรือ Strategy เลยไม่กระทบ Controller โดยตรง ช่วยให้ระบบดูแลรักษาและขยายต่อได้ง่าย

4.Information Expert

หลัก Information Expert ถูกนำมาใช้ในการกำหนดว่าคลาสไหนควรรับผิดชอบงานใด เช่น GameRepository เป็นการเข้าถึงข้อมูลจากฐานข้อมูลเพราะมีข้อมูลการเชื่อมต่อ ในขณะที่ DiscountContext ในการคำนวณราคาส่วนลด มี Strategy ทั้งหมดไว้ภายใน

5.Indirection

การที่ Controller ไม่เรียก Repository โดยตรง แต่เรียกผ่าน Service Layer เป็นการใช้หลัก Indirection เพื่อสร้างชั้นกลางที่ลดการพึ่งพากันโดยตรงระหว่าง Presentation Layer และ Data Access Layer ทำให้สามารถเปลี่ยนแปลงวิธีการเข้าถึงข้อมูลได้โดยไม่กระทบ Controller

1.2 High-Level SOLID Principles

หลักการ	การประยุกต์ใช้ในระบบ
S — Single Responsibility	แต่ละคลาสมีหน้าที่เดียว เช่น Game เก็บข้อมูล, GameRepository เข้าถึงข้อมูล, GameService จัดการ business logic, GameController จัดการ HTTP request/response
O — Open/Closed	Strategy Pattern ทำให้เพิ่มส่วนลดรูปแบบใหม่ได้โดยสร้างคลาส implement DiscountStrategy เพิ่มโดยไม่ต้องแก้ไขโค้ดเดิมของ NoDiscountStrategy, StudentDiscountStrategy หรือ SeasonalSaleStrategy
L — Liskov Substitution	ทุกคลาสที่ implement DiscountStrategy (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) สามารถใช้แทนกันได้ผ่าน interface เดียวกันโดยไม่ทำให้ระบบทำงานผิดพลาด
I — Interface Segregation	DiscountStrategy เป็น interface ขนาดเล็กมี method เดียวคือ applyDiscount() ไม่บังคับให้คลาสที่ implement ต้องมี method ที่ไม่เกี่ยวข้อง

หลักการ	การประยุกต์ใช้ในระบบ
D — Dependency Inversion	GameController และ GameService ขึ้นกับ abstraction (interface/Service) ไม่ได้ผูกกับ implementation โดยตรง และรับ dependency ผ่าน Constructor Injection ทำให้ high-level module ไม่ขึ้นกับ low-level module โดยตรง

1.3 Strategy Pattern

ระบบใช้ Strategy Pattern ในการคำนวณราคาเกมหลังหักส่วนลด โดยมีโครงสร้างดังนี้

- DiscountStrategy เป็น interface กลาง กำหนด method applyDiscount(double originalPrice) เป็นสัญญาที่ทุก strategy ต้องปฏิบัติตาม
- NoDiscountStrategy ไม่มีส่วนลด คำนวณราคาปกติ
- StudentDiscountStrategy ลดราคา 10% สำหรับส่วนลดนักศึกษา
- SeasonalSaleStrategy ลดราคา 20% สำหรับโปรโมชั่นเทศกาล
- DiscountContext ตัวกลางที่เลือก Strategy ที่เหมาะสมตามค่า discountType แล้วเรียกใช้ applyDiscount() เพื่อคำนวณราคาสุทธิ

การออกแบบนี้ทำให้ตรรกะการคำนวณส่วนลดแต่ละแบบถูกแยกออกจากกันอย่างชัดเจน (Single Responsibility) และถ้าในอนาคตต้องการเพิ่มส่วนลดรูปแบบใหม่ก็แค่เพียงสร้างคลาสใหม่ implement DiscountStrategy และเพิ่มเงื่อนไขใน DiscountContext ได้เลยโดยไม่ต้องแก้ไขโค้ดของ Strategy เดิมที่มีอยู่แล้ว ซึ่งสอดคล้องกับหลัก Open/Closed Principle (OCP) โดยตรง

1.4 Layered Architecture

ระบบถูกแบ่งออกเป็น 4 ชั้น (Layer) ตามลำดับการทำงาน ดังนี้

ชั้น (Layer)	หน้าที่รับผิดชอบ
Controller Layer	รับ HTTP Request, ส่งต่อให้ Service, เลือก View หรือ Redirect
Service Layer	จัดการ Business Logic, เรียกใช้ Strategy Pattern เพื่อคำนวณราคา, ประสานงานกับ Repository
Repository Layer	เข้าถึงและจัดการข้อมูลในฐานข้อมูลผ่าน Spring Data JPA
Entity/Model Layer	แทนโครงสร้างข้อมูล และ map กับตารางในฐานข้อมูล

เหตุผลที่ต้องแยก Service Layer ออกจาก Controller และ Repository คือเพื่อให้เกิด Low Coupling และ High Cohesion กล่าวคือ Controller ไม่ต้องรู้รายละเอียดว่าข้อมูลถูกเก็บหรือคำนวณอย่างไร มีหน้าที่เพียงส่งต่อคำขอ ในขณะที่ Repository ก็ไม่ต้องรู้ว่าจะถูกเรียกใช้จากที่ใด การแยกเช่นนี้ทำให้แต่ละ Layer สามารถถูกทดสอบ (Unit Test) แยก หรือแทนที่ได้ง่ายอิสระ โดยไม่กระทบ Layer อื่น อีกทั้งยังง่ายต่อการขยายระบบในอนาคต เช่น การเพิ่ม business rule ใหม่จะเฉพาะที่ Service เท่านั้น

1.5 Execution Flow

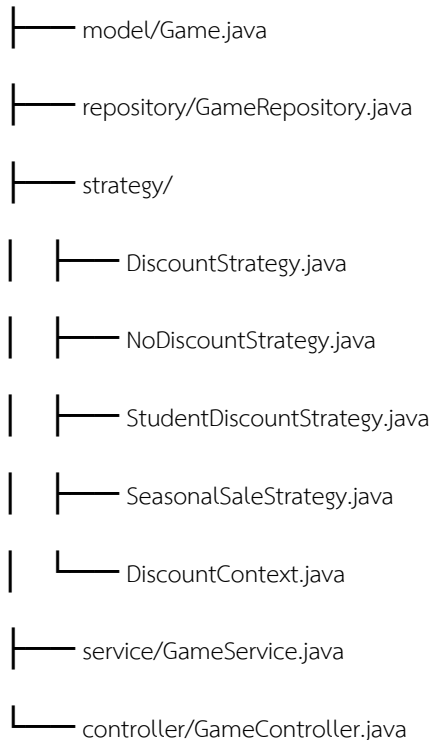
ลำดับการทำงานของระบบเมื่อมี HTTP Request เข้ามาจนถึงการบันทึกข้อมูลลง PostgreSQL และคำนวณส่วนลด มีดังนี้ (ยกตัวอย่างกรณีเพิ่มเกมใหม่ผ่านฟอร์ม Add):

1. ผู้ใช้กรอกฟอร์มในหน้า add.html แล้วกดปุ่ม "บันทึก" ซึ่งจะส่ง HTTP POST ไปยัง /games/save
2. DispatcherServlet ของ Spring รับ Request แล้วส่งต่อให้ GameController ตาม @PostMapping("/save") ที่กำหนดไว้
3. Spring ทำ Data Binding ค่าจากฟอร์มเข้าสู่ Object Game ผ่าน @ModelAttribute โดยอัตโนมัติ
4. GameController เรียกใช้ gameService.saveGame(game) โดยไม่ทราบรายละเอียดว่าข้อมูลจะถูกบันทึกอย่างไร (Indirection)
5. GameService เรียกใช้ gameRepository.save(game) เพื่อบันทึกข้อมูลลงฐานข้อมูล
6. GameRepository (ซึ่ง extends JpaRepository) แปลงคำสั่งเป็น SQL ผ่าน Hibernate แล้วส่งไปบันทึกยัง PostgreSQL จริง
7. เมื่อบันทึกสำเร็จ Controller จะ redirect กลับไปที่ /games พร้อมข้อความแจ้งเตือนผ่าน RedirectAttributes
8. เมื่อ Browser ยิง GET /games ตามมา Controller จะเรียก gameService.getAllGames() ซึ่งไปเรียก gameRepository.findAll() เพื่อดึงข้อมูลทั้งหมดจากฐานข้อมูล
9. สำหรับแต่ละเกม ค่าราคาสุทธิ (finalPrice) จะถูกคำนวณผ่าน Strategy Pattern โดยพิจารณาจาก discountType ของเกมนั้น ๆ
10. Controller ส่งรายการเกมทั้งหมดไปยัง Model แล้วเลือก View เป็น games/list ให้ Thymeleaf Engine render เป็นหน้า HTML กลับไปแสดงผลที่ Browser

ส่วนที่ 2: Code Implementation & Explanation

โครงสร้างโปรเจกต์แบ่งตาม Layered Architecture ดังนี้

src/main/java/com/example/lab7_673380594_9/



2.1 Entity : Game.java

Game เป็น Entity ที่ map กับตาราง games ในฐานข้อมูล PostgreSQL โดยใช้ Annotation ของ JPA ได้แก่ @Entity, @Table, @Id และ @GeneratedValue(strategy = GenerationType.IDENTITY) เพื่อให้ PostgreSQL สร้างค่า id แบบ auto-increment ให้อัตโนมัติ นอกจากนี้ฟิลด์พื้นฐาน (title, genre, platform, rating, releaseDate, price, discountType) แล้ว ยังมี derived property สองตัวคือ getDiscountName() ที่แปลงรหัสส่วนลดให้อ่านง่าย และ getFinalPrice() ที่คำนวณราคาสุทธิสำหรับแสดงผลในหน้า list

2.2 Repository :GameRepository.java

```
@Repository
public interface GameRepository extends JpaRepository<Game, Long> {
}
```

GameRepository เพียง extend JpaRepository<Game, Long> ก็ได้ CRUD พื้นฐานครบถ้วนโดยไม่ต้องเขียน SQL เอง Spring Data JPA จะ generate implementation ให้อัตโนมัติในขณะ runtime

2.3 Strategy Package

แพ็คเกจ strategy ประกอบด้วย interface DiscountStrategy และคลาสที่ implement 3 แบบ ได้แก่ NoDiscountStrategy, StudentDiscountStrategy (ลด 10%) และ SeasonalSaleStrategy (ลด 20%) โดยทุกคลาสถูกประกาศเป็น @Component เพื่อให้ Spring

จัดการเป็น Bean ส่วน DiscountContext ทำหน้าที่รับทั้ง 3 Strategy เข้ามาผ่าน Constructor Injection แล้วเลือกใช้ตาม discountType ที่ส่งเข้ามาผ่าน method calculatePrice()

2.4 Service: GameService.java

GameService รับ GameRepository และ DiscountContext ผ่าน Constructor Injection ทำหน้าที่เป็นศูนย์กลาง Business Logic ของระบบ ได้แก่การดึงข้อมูลทั้งหมด (getAllGames), ค้นหาตาม id (getGameById), เพิ่มข้อมูล (saveGame), แก้ไขข้อมูล (updateGame), ลบข้อมูล (deleteGame) และคำนวณราคาสุทธิผ่าน Strategy Pattern (calculateFinalPrice) การแยก Layer นี้ทำให้ Controller ไม่ต้องยุ่งเกี่ยวกับรายละเอียดการคำนวณหรือการเข้าถึงข้อมูลโดยตรง

2.5 Controller : GameController.java

GameController รับ GameService ผ่าน Constructor Injection เพียงตัวเดียว และเปิดให้บริการ endpoint ครบ 7 เส้นทางตามที่ระบบต้องการ ได้แก่ แสดงรายการ (/games), แสดงฟอร์มเพิ่ม (/games/add), บันทึกข้อมูลใหม่ (/games/save), แสดงฟอร์มแก้ไข (/games/edit/{id}), อัปเดตข้อมูล (/games/update/{id}), แสดงหน้ายืนยันลบ และลบข้อมูลจริง (/games/delete/{id}) โดยใช้ RedirectAttributes ในการส่งข้อความแจ้งเตือนผ่าน redirect เพื่อหลีกเลี่ยงปัญหาการเข้ารหัสอักขระภาษาไทยใน URL

การใช้ Constructor Injection ในทุก Layer (Controller → Service → Repository/Strategy) แทนการใช้ @Autowired บน field โดยตรง ช่วยให้ dependency ของแต่ละคลาสถูกกำหนดอย่างชัดเจนตั้งแต่การสร้าง object (immutability), ช่วยต่อการเขียน Unit Test ด้วยการ mock dependency ผ่าน constructor และสอดคล้องกับหลัก Dependency Inversion Principle (DIP) ที่ high-level module ควรขึ้นกับ abstraction มากกว่า concrete implementation

ส่วนที่ 3: Web Application & Database Screenshots

3.1 หน้าจอเพิ่มเกมใหม่ (Create)

ภาพหน้าจอฟอร์มเพิ่มเกมที่กำลังกรอกข้อมูล โดยต้องใส่รหัสนักศึกษาและ Section ลงในชื่อเกม (Title) หรือแนวเกม (Genre) เช่น Elden Ring (673380594-9 SEC ...) — แนวภาพทั้งกรณีส่วนลดนักศึกษา (10%) และส่วนลดเทศกาล (20%)

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

เพิ่มเกมใหม่
กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม
No time to relax (Lab7-67380594-9-sec)

แนวเกม (Genre) Platform (Platform)
RPG PC

คะแนน (Rating) ราคาปกติ (บาท) % ส่วนลด (Strategy)
10.0 399 ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)
01/08/2026

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)

ภาพหน้าจอ list ที่เห็นแถบแจ้งเตือนสีเขียวหลังบันทึกสำเร็จ และข้อมูลเกมที่มีรหัสนักศึกษาปรากฏอยู่ในตาราง

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

เพิ่มเกมใหม่

เพิ่มเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	III	etc	PC	★ 9.2	13/08/2026	ส่วนลดเทศกาล (20%)	98.40 123.00	แก้ไข ลบ
2	No time to relax (Lab7-67380594-9-sec3)	RPG	PC	★ 10.0	01/08/2026	ส่วนลดนักศึกษา (10%)	359.10 399.00	แก้ไข ลบ

ทั้งหมด 2 เกม

3.3 หน้าจอแก้ไขเกม (Update)

ภาพหน้าจอฟอร์มแก้ไขข้อมูลเกมที่มีอยู่แล้ว

← → ↻ localhost:8080/games/edit/6 ☆ 📁 🌐 | Google 📁 จักรวาลเกม

Game Catalog CP353002 — Lab 7

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: No time to relax (Lab7-67380594-9-sec3)

ชื่อเกม
No time to relax (Lab7-67380594-9-sec3)

แนวเกม (Genre)
RPG

แพลตฟอร์ม (Platform)
PC

คะแนน (Rating)
10.0

ราคาปกติ (บาท)
500.0

% ส่วนลด (Strategy)
ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)
วว/ดด/ปปปป

✅ อัปเดต ✕ ยกเลิก

← → ↻ localhost:8080/games ☆ 📁 🌐 | Google 📁 จักรวาลเกม

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+ เพิ่มเกมใหม่

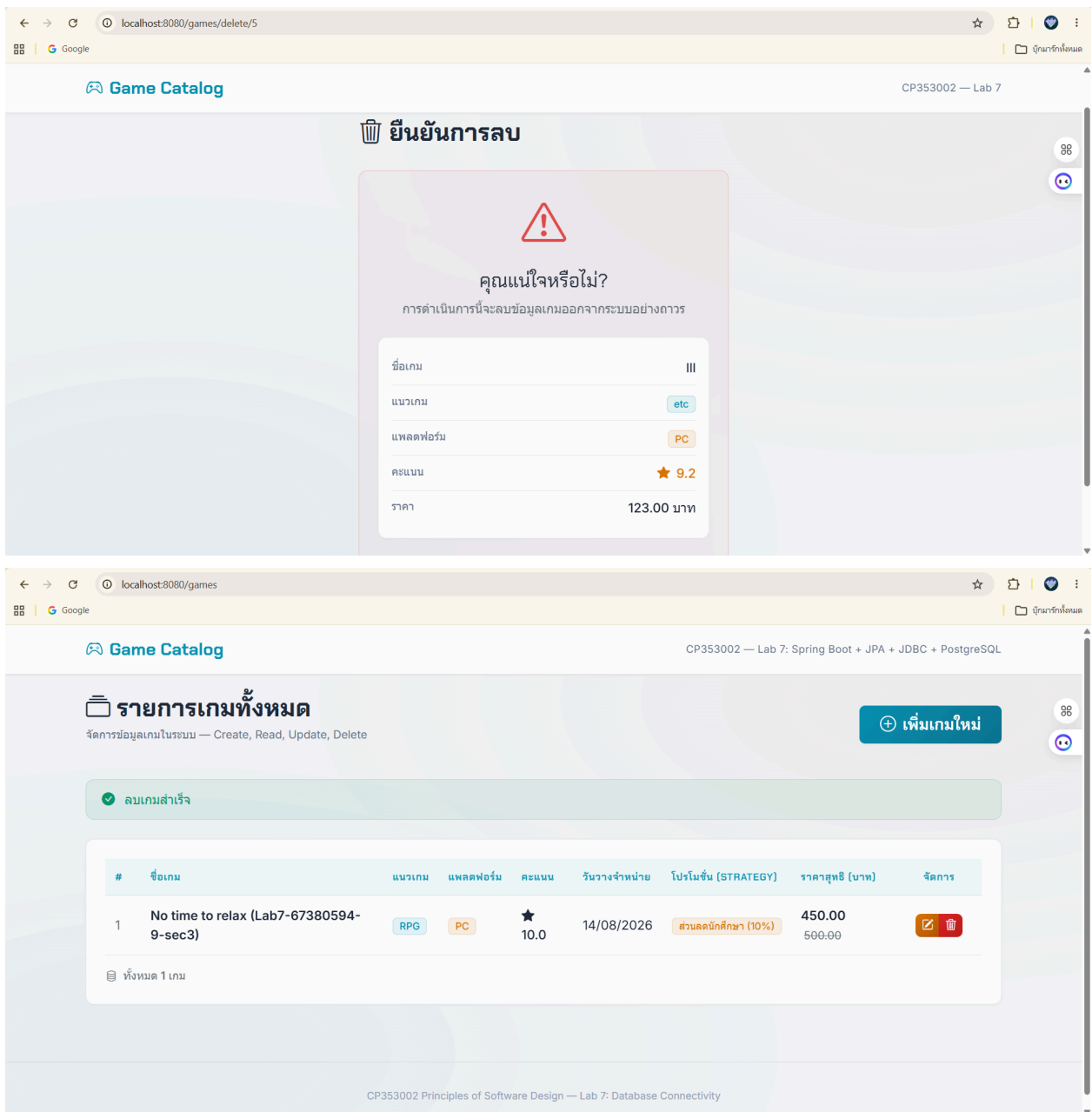
✅ แก้ไขเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	III	etc	PC	★ 9.2	13/08/2026	ส่วนลดเทศกาล (20%)	98.40 123.00	✎ 🗑
2	No time to relax (Lab7-67380594-9-sec3)	RPG	PC	★ 10.0	14/08/2026	ส่วนลดนักศึกษา (10%)	450.00 500.00	✎ 🗑

📄 ทั้งหมด 2 เกม

3.4 หน้าจอยืนยันลบและผลลัพธ์ (Delete)

ภาพหน้าจอยืนยันการลบ และภาพหลังจากลบสำเร็จที่กลับไปหน้ารายการ



3.5 หน้าจอฐานข้อมูล (pgAdmin / psql)

ภาพหน้าจอ pgAdmin หรือ terminal psql ที่แสดงข้อมูลจริงในตาราง games ซึ่งมีรหัสนักศึกษามันที่ก้อยู่ในฐานข้อมูล

pgAdmin 4

FileObjectToolsEditViewWindowHelp

Object Explorer

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (1)

games

Columns

Constraints (1)

games_pkey

Indexes

RLS Policies

Rules

Triggers

Trigger Functions

Types

Views

Subscriptions

postgres

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Dashboard

Properties

SQL

Statistics

Dependencies

Dependents

Processes

public.games/lab7demo/postgres@PostgreSQL 18

public.games/lab7demo/postgres@PostgreSQL 18

No limit

Query

Query History

Scratch Pad

1

SELECT * FROM public.games

2

ORDER BY id ASC

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

id	discount_type	genre	platform	price	rating	release_date	title	
[PK] bigint	character varying (255)	character varying (255)	character varying (255)	double precision	double precision	date	character varying (255)	
1	6	STUDENT	RPG	PC	500	10	2026-08-14	No time to relax (Lab7-67380594-9-se...

Total rows: 1

Query complete 00:00:00.096

CRLF

Ln 2, Col 17